

COMP214 – Advanced Database Concepts

Term Project Report

1. Group Members

Team Name: ECommerceTeam

- Chun Wai Chung 300123456
 - Burak Can Koseoglu 301217347
 - HyojinKim 303177324
 - Brian Bernales 301489703
-

2. Problem Domain Description

This project implements a comprehensive backend for an e-commerce system using Oracle Database and PL/SQL programming units.

The database system manages customers, products, shopping carts, order items, payments, and shipping processes.

It supports customer registration, product and inventory management, the checkout process, payment tracking, and shipping status.

The primary goal is to ensure efficient, reliable, and maintainable data storage and business logic for a real-world online shopping platform.

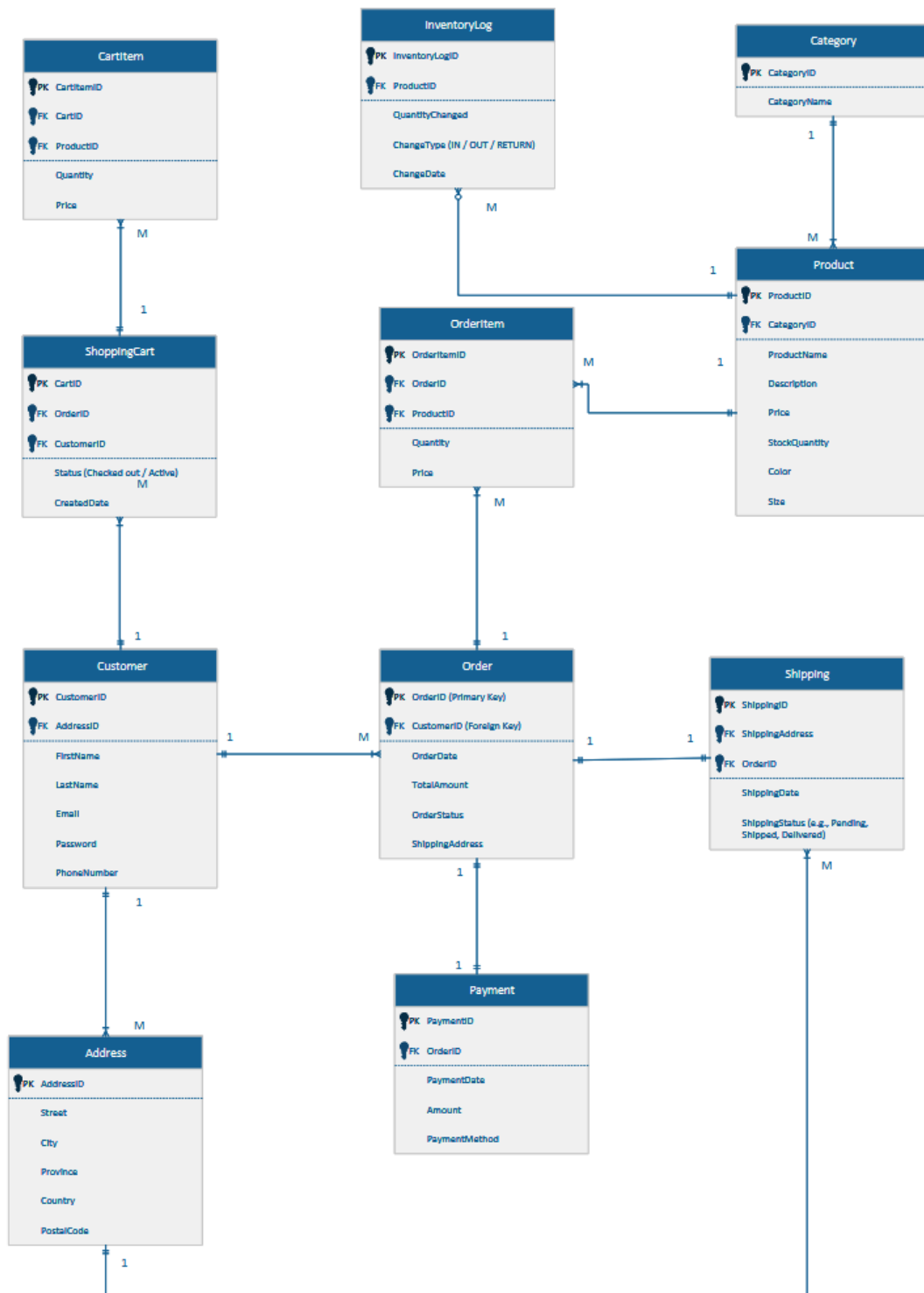
3. ER Diagram

Description:

The ER diagram shows all main tables and their relationships:

- Customers can create multiple shopping carts.
- Each cart contains one or more order items.
- Payments are made per cart, and shipping is tracked per cart.
- Products are referenced in both order items and shopping carts.

3. ER Diagram



4. Screenshots: Sample Table Content (Data Viewing)

4.1. Comm_customer

```
CREATE TABLE comm_customer (  
  customer_id NUMBER PRIMARY KEY,  
  first_name VARCHAR2(50) NOT NULL,  
  last_name VARCHAR2(50) NOT NULL,  
  email VARCHAR2(100) UNIQUE NOT NULL,  
  phone VARCHAR2(20),  
  address VARCHAR2(255) NOT NULL  
);  
  
INSERT INTO comm_customer VALUES (1, 'Alice', 'Smith', 'alice@gmail.com', '1234567890', '22 Front St, Toronto');  
INSERT INTO comm_customer VALUES (2, 'Bob', 'Johnson', 'bob.j@email.com', '2345678901', '45 Oak Ave, Vancouver');  
INSERT INTO comm_customer VALUES (3, 'Charlie', 'Brown', 'charlie.b@email.com', '3456789012', '78 Pine Rd, Montreal');  
INSERT INTO comm_customer VALUES (4, 'Diana', 'Miller', 'diana.m@email.com', '4567890123', '101 Elm St, Calgary');  
INSERT INTO comm_customer VALUES (5, 'Ethan', 'Davis', 'ethan.d@email.com', '5678901234', '202 Maple Dr, Halifax');  
INSERT INTO comm_customer VALUES (6, 'Fiona', 'Wilson', 'fiona.w@email.com', '6789012345', '303 Cedar Ln, Ottawa');  
INSERT INTO comm_customer VALUES (7, 'George', 'Martinez', 'george.m@email.com', '7890123456', '404 Birch Blvd, Edmonton');  
INSERT INTO comm_customer VALUES (8, 'Hannah', 'Anderson', 'hannah.a@email.com', '8901234567', '505 Spruce Way, Winnipeg');  
INSERT INTO comm_customer VALUES (9, 'Ian', 'Thomas', 'ian.t@email.com', '9012345678', '606 Willow Cres, Quebec City');  
INSERT INTO comm_customer VALUES (10, 'Julia', 'White', 'julia.w@email.com', '0123456789', '707 Aspen Ct, Victoria');
```

CUSTOMER_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE	ADDRESS
1	Alice	Smith	alice@gmail.com	1234567890	22 Front St, Toronto
2	Bob	Johnson	bob.j@email.com	2345678901	45 Oak Ave, Vancouver
3	Charlie	Brown	charlie.b@email.com	3456789012	78 Pine Rd, Montreal
4	Diana	Miller	diana.m@email.com	4567890123	101 Elm St, Calgary
5	Ethan	Davis	ethan.d@email.com	5678901234	202 Maple Dr, Halifax
6	Fiona	Wilson	fiona.w@email.com	6789012345	303 Cedar Ln, Ottawa
7	George	Martinez	george.m@email.com	7890123456	404 Birch Blvd, Edmonton
8	Hannah	Anderson	hannah.a@email.com	8901234567	505 Spruce Way, Winnipeg
9	Ian	Thomas	ian.t@email.com	9012345678	606 Willow Cres, Quebec City
10	Julia	White	julia.w@email.com	0123456789	707 Aspen Ct, Victoria

▪ Description:

Stores customer details such as name, email, phone number, and address.

4.2. Comm_product

```
CREATE TABLE comm_product (  
  product_id NUMBER PRIMARY KEY,  
  category_id NUMBER NOT NULL,  
  product_name VARCHAR2(100) NOT NULL,  
  color VARCHAR2(30),  
  price NUMBER(10,2) NOT NULL,  
  stock_quantity NUMBER NOT NULL  
);  
  
INSERT INTO comm_product VALUES (1, 1, 'Wireless Headphones', 'Black', 89.99, 50);  
INSERT INTO comm_product VALUES (2, 1, 'Bluetooth Speaker', 'Blue', 59.99, 30);  
INSERT INTO comm_product VALUES (3, 2, 'Cotton T-Shirt', 'White', 19.99, 100);  
INSERT INTO comm_product VALUES (4, 2, 'Denim Jeans', 'Blue', 49.99, 75);  
INSERT INTO comm_product VALUES (5, 3, 'Coffee Maker', 'Silver', 79.99, 20);  
INSERT INTO comm_product VALUES (6, 3, 'Toaster', 'Black', 39.99, 40);  
INSERT INTO comm_product VALUES (7, 4, 'Running Shoes', 'Red', 89.99, 60);  
INSERT INTO comm_product VALUES (8, 4, 'Yoga Mat', 'Purple', 29.99, 90);  
INSERT INTO comm_product VALUES (9, 5, 'Backpack', 'Gray', 49.99, 45);  
INSERT INTO comm_product VALUES (10, 5, 'Laptop Bag', 'Black', 59.99, 35);
```

PRODUCT_ID	CATEGORY_ID	PRODUCT_NAME	COLOR	PRICE	STOCK_QUANTITY
1	1	Wireless Headphones	Black	89.99	50
2	1	Bluetooth Speaker	Blue	59.99	30
3	2	Cotton T-Shirt	White	19.99	100
4	2	Denim Jeans	Blue	49.99	75
5	3	Coffee Maker	Silver	79.99	20
6	3	Toaster	Black	39.99	40
7	4	Running Shoes	Red	89.99	60
8	4	Yoga Mat	Purple	29.99	90
9	5	Backpack	Gray	49.99	45
10	5	Laptop Bag	Black	59.99	35

▪ Description

Stores product details, including category, price, color, and stock quantity.

4.3. Comm_shopping_cart

```
CREATE TABLE comm_shopping_cart (
  cart_id NUMBER PRIMARY KEY,
  customer_id NUMBER NOT NULL,
  product_id NUMBER NOT NULL,
  quantity NUMBER NOT NULL,
  price_each NUMBER(10,2) NOT NULL,
  status VARCHAR2(20) CHECK (status IN ('Checked Out', 'Active')),
  FOREIGN KEY (customer_id) REFERENCES comm_customer(customer_id),
  FOREIGN KEY (product_id) REFERENCES comm_product(product_id)
);

INSERT INTO comm_shopping_cart VALUES (1, 1, 1, 2, 89.99, 'Checked Out');
INSERT INTO comm_shopping_cart VALUES (2, 2, 3, 1, 19.99, 'Checked Out');
INSERT INTO comm_shopping_cart VALUES (3, 3, 5, 1, 79.99, 'Active');
INSERT INTO comm_shopping_cart VALUES (4, 4, 7, 1, 89.99, 'Checked Out');
INSERT INTO comm_shopping_cart VALUES (5, 5, 9, 1, 49.99, 'Active');
INSERT INTO comm_shopping_cart VALUES (6, 6, 2, 3, 59.99, 'Checked Out');
INSERT INTO comm_shopping_cart VALUES (7, 7, 4, 2, 49.99, 'Active');
INSERT INTO comm_shopping_cart VALUES (8, 8, 6, 1, 39.99, 'Checked Out');
INSERT INTO comm_shopping_cart VALUES (9, 9, 8, 1, 29.99, 'Active');
INSERT INTO comm_shopping_cart VALUES (10, 10, 10, 1, 59.99, 'Checked Out');
```

스크립트 출력 x

작업이 완료되었습니다.(0.634초)

```
-- CART_ID CUSTOMER_ID PRODUCT_ID -- QUANTITY PRICE_EACH STATUS -----
-----
----- 1 ----- 1 ----- 1 ----- 2 ----- 89.99 Checked Out -----
----- 2 ----- 2 ----- 3 ----- 1 ----- 19.99 Checked Out -----
----- 3 ----- 3 ----- 5 ----- 1 ----- 79.99 Active -----
----- 4 ----- 4 ----- 7 ----- 1 ----- 89.99 Checked Out -----
----- 5 ----- 5 ----- 9 ----- 1 ----- 49.99 Active -----
----- 6 ----- 6 ----- 2 ----- 3 ----- 59.99 Checked Out -----
----- 7 ----- 7 ----- 4 ----- 2 ----- 49.99 Active -----
----- 8 ----- 8 ----- 6 ----- 1 ----- 39.99 Checked Out -----
----- 9 ----- 9 ----- 8 ----- 1 ----- 29.99 Active -----
----- 10 ----- 10 ----- 10 ----- 1 ----- 59.99 Checked Out -----
```

▪ Description

Stores shopping cart records, linking customers with products, quantities, and status.

4.4. Comm_payment

```
CREATE TABLE comm_payment (
  payment_id NUMBER PRIMARY KEY,
  cart_id NUMBER NOT NULL,
  payment_method VARCHAR2(50),
  payment_status VARCHAR2(20) CHECK (payment_status IN ('Paid', 'Pending', 'Failed')),
  FOREIGN KEY (cart_id) REFERENCES comm_shopping_cart(cart_id)
);

INSERT INTO comm_payment VALUES (1, 1, 'Credit Card', 'Paid');
INSERT INTO comm_payment VALUES (2, 2, 'PayPal', 'Paid');
INSERT INTO comm_payment VALUES (3, 3, 'Credit Card', 'Pending');
INSERT INTO comm_payment VALUES (4, 4, 'Debit Card', 'Paid');
INSERT INTO comm_payment VALUES (5, 5, 'Apple Pay', 'Pending');
INSERT INTO comm_payment VALUES (6, 6, 'Credit Card', 'Paid');
INSERT INTO comm_payment VALUES (7, 7, 'Google Pay', 'Pending');
INSERT INTO comm_payment VALUES (8, 8, 'Credit Card', 'Paid');
INSERT INTO comm_payment VALUES (9, 9, 'PayPal', 'Failed');
INSERT INTO comm_payment VALUES (10, 10, 'Debit Card', 'Paid');
```

```

PAYMENT_ID  CART_ID  PAYMENT_METHOD  PAYMENT_STATUS
-----
1          1  Credit Card    Paid
2          2   PayPal       Paid
3          3  Credit Card    Pending
4          4  Debit Card     Paid
5          5   Apple Pay    Pending
6          6  Credit Card    Paid
7          7   Google Pay   Pending
8          8  Credit Card    Paid
9          9   PayPal       Failed
10         10  Debit Card     Paid
```

• Description

Stores payment information for each cart, including method and status.

4.5. Comm_shipping

```
CREATE TABLE comm_shipping (  
  shipping_id NUMBER PRIMARY KEY,  
  cart_id NUMBER NOT NULL,  
  shipping_address VARCHAR2(255),  
  shipping_status VARCHAR2(20) CHECK (shipping_status IN ('Pending', 'Shipped', 'Delivered')),  
  FOREIGN KEY (cart_id) REFERENCES comm_shopping_cart (cart_id)  
);  
  
INSERT INTO comm_shipping VALUES (1, 1, '22 Front St, Toronto', 'Delivered');  
INSERT INTO comm_shipping VALUES (2, 2, '45 Oak Ave, Vancouver', 'Shipped');  
INSERT INTO comm_shipping VALUES (3, 3, '78 Pine Rd, Montreal', 'Pending');  
INSERT INTO comm_shipping VALUES (4, 4, '101 Elm St, Calgary', 'Delivered');  
INSERT INTO comm_shipping VALUES (5, 5, '202 Maple Dr, Halifax', 'Pending');  
INSERT INTO comm_shipping VALUES (6, 6, '303 Cedar Ln, Ottawa', 'Shipped');  
INSERT INTO comm_shipping VALUES (7, 7, '404 Birch Blvd, Edmonton', 'Pending');  
INSERT INTO comm_shipping VALUES (8, 8, '505 Spruce Way, Winnipeg', 'Delivered');  
INSERT INTO comm_shipping VALUES (9, 9, '606 Willow Cres, Quebec City', 'Pending');  
INSERT INTO comm_shipping VALUES (10, 10, '707 Aspen Ct, Victoria', 'Shipped');  
  
SELECT * FROM comm_customer;  
SELECT * FROM comm_product;  
SELECT * FROM comm_shopping_cart;  
SELECT * FROM comm_payment;  
SELECT * FROM comm_shipping;
```

SHIPPING_ID	CART_ID	SHIPPING_ADDRESS	SHIPPING_STATUS
1	1	22 Front St, Toronto	Delivered
2	2	45 Oak Ave, Vancouver	Shipped
3	3	78 Pine Rd, Montreal	Pending
4	4	101 Elm St, Calgary	Delivered
5	5	202 Maple Dr, Halifax	Pending
6	6	303 Cedar Ln, Ottawa	Shipped
7	7	404 Birch Blvd, Edmonton	Pending
8	8	505 Spruce Way, Winnipeg	Delivered
9	9	606 Willow Cres, Quebec City	Pending
10	10	707 Aspen Ct, Victoria	Shipped

▪ Description

Stores shipping status and address for each cart.

5. Screenshots: Programming Units

5.1. Sequences

: customer_seq / payment_seq

Purpose: Generates unique IDs for customer and payment records to ensure consistent and automatic primary key assignment.

```
<<f
CREATE SEQUENCE customer_seq START WITH 11 INCREMENT BY 1 NOCACHE;<<f
<<f
INSERT INTO comm_customer<<f
  ..(customer_id, first_name, last_name, email, phone, address)<<f
VALUES<<f
  ..(customer_seq.NEXTVAL, 'Kevin', 'Lee', 'kevin@email.com', '01012345678', '123 Queen St, Toronto');<<f
<<f
SELECT * FROM comm_customer WHERE email = 'kevin@email.com';<<f
<<f
```

Sequence CUSTOMER_SEQ이 (가) 생성되었습니다.<<f

<<f

<<f

1 행 이 (가) 삽입되었습니다.<<f

..

	CUSTOMER_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE	ADDRESS	
1	11	Kevin	Lee	kevin@email.com	01012345678	123 Queen St, Toronto	

```
INSERT INTO comm_payment (payment_id, cart_id, payment_method, payment_status)<<f
VALUES (payment_seq.NEXTVAL, 3, 'Credit Card', 'Paid');<<f
<<f
CREATE SEQUENCE payment_seq START WITH 11 INCREMENT BY 1 NOCACHE;<<f
<<f
```

Sequence PAYMENT_SEQ이 (가) 생성되었습니다.<<f

<<f

<<f

1 행 이 (가) 삽입되었습니다.<<f

..

	PAYMENT_ID	CART_ID	PAYMENT_METHOD	PAYMENT_STATUS	
1	11	3	Credit Card	Paid	
2	3	3	Credit Card	Pending	

5.2. Index

5.2.1 Index : idx_customer_email

- Target Table: comm_customer
- Column Indexed: email
- Purpose: Speeds up customer search by email.

```
CREATE INDEX idx_customer_email ON comm_customer(email);  
  
SELECT * FROM comm_customer WHERE email = 'alice@gmail.com';
```

	CUSTOMER_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE	ADDRESS
1	1	Alice	Smith	alice@gmail.com	1234567890	22 Front St, Toronto

5.2.2 Index : idx_product_name

- Target Table: comm_product
- Column Indexed: product_name
- Purpose: Speeds up product search by name, especially for filtering and listing.

```
CREATE INDEX idx_product_name ON comm_product(product_name);  
  
SELECT * FROM comm_product WHERE product_name LIKE '%Speaker%';
```

Index 'IDX_PRODUCT_NAME' (가) 생성되었습니다.

	PRODUCT_ID	CATEGORY_ID	PRODUCT_NAME	COLOR	PRICE	STOCK_QUANTITY
1	2	1	Bluetooth Speaker	Blue	59.99	30

5.3. Triggers

5.3.1 Trigger : trg_update_product_stock

- Target Table: comm_shopping_cart
- Event: AFTER UPDATE OF status
- Condition: NEW.status = 'Checked Out' AND OLD.status = 'Active'
- Purpose: Reduces product stock when checkout is completed.

```
CREATE OR REPLACE TRIGGER trg_update_product_stock«¶
AFTER UPDATE OF status ON comm_shopping_cart«¶
FOR EACH ROW«¶
WHEN (NEW.status = 'Checked Out' AND OLD.status = 'Active') «¶
BEGIN«¶
    ... UPDATE comm_product«¶
    ... SET stock_quantity = stock_quantity - :NEW.quantity«¶
    ... WHERE product_id = :NEW.product_id;«¶
«¶
    ... DBMS_OUTPUT.PUT_LINE('Product stock updated. ');«¶
END;«¶
/«¶
«¶
UPDATE comm_shopping_cart«¶
SET status = 'Checked Out'«¶
WHERE cart_id = 3;«¶
«¶
SELECT stock_quantity FROM comm_product WHERE product_id = 5;«¶
~»
```

Trigger TRG_UPDATE_PRODUCT_STOCK이 (가) 컴파일되었습니다.«¶

«¶

«¶

1 행 이 (가) 업데이트되었습니다.«¶

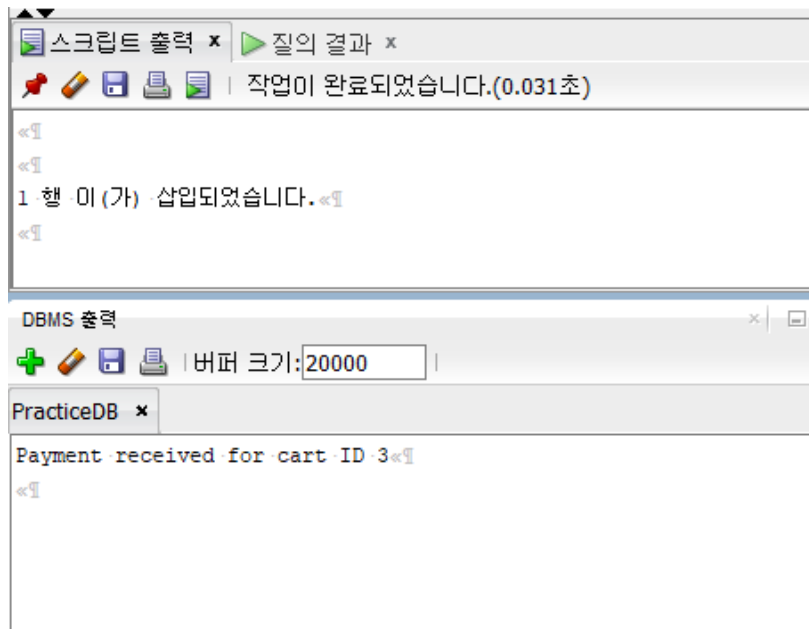
«¶

	STOCK_QUANTITY
1	19

5.3.2 Trigger : trg_update_customer_stats

- Target Table: comm_payment
- Event: AFTER INSERT OR UPDATE OF payment_status
- Condition: NEW.payment_status = 'Paid'
- Purpose: Logs message when payment is marked as paid.

```
CREATE OR REPLACE TRIGGER trg_update_customer_stats«¶
AFTER INSERT OR UPDATE OF payment_status ON comm_payment«¶
FOR EACH ROW«¶
WHEN (NEW.payment_status = 'Paid') «¶
BEGIN«¶
  ...DBMS_OUTPUT.PUT_LINE('Payment received for cart ID ' || :NEW.cart_id);«¶
END;«¶
/«¶
«¶
INSERT INTO comm_payment (payment_id, cart_id, payment_method, payment_status)«¶
VALUES (payment_seq.NEXTVAL, 3, 'Credit Card', 'Paid');«¶
«¶
```

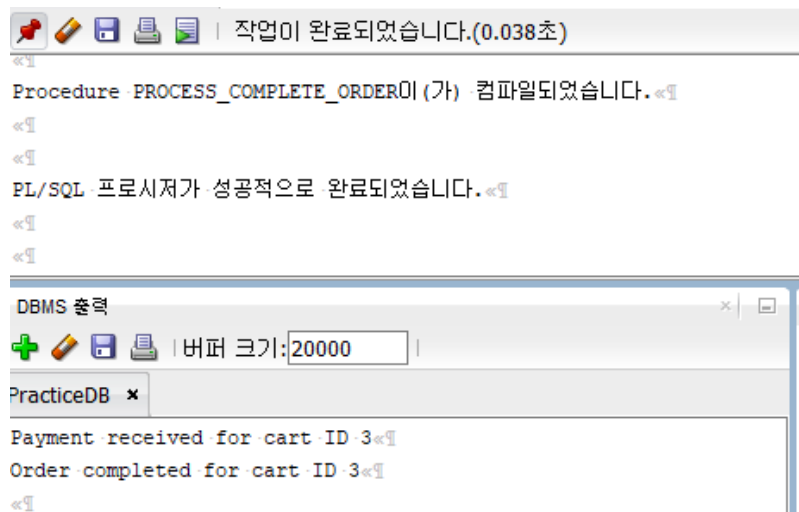


5.4. Procedures

5.4.1 Procedure : process_complete_order

- Purpose: Completes the order by updating cart status and creating a payment record.

```
CREATE OR REPLACE PROCEDURE process_complete_order (  
    ... p_cart_id IN NUMBER, <¶  
    ... p_method IN VARCHAR2 <¶  
) AS <¶  
BEGIN <¶  
    ... -- Update cart status <¶  
    ... UPDATE comm_shopping_cart <¶  
    ... SET status = 'Checked Out' <¶  
    ... WHERE cart_id = p_cart_id; <¶  
<¶  
    ... -- Insert payment <¶  
    ... INSERT INTO comm_payment (  
    ... payment_id, cart_id, payment_method, payment_status <¶  
    ... ) VALUES (  
    ... payment_seq.NEXTVAL, p_cart_id, p_method, 'Paid' <¶  
    ... ); <¶  
<¶  
    ... DBMS_OUTPUT.PUT_LINE('Order completed for cart ID-' || p_cart_id); <¶  
END; <¶  
/<¶  
<¶  
BEGIN <¶  
    ... process_complete_order(3, 'Credit Card'); <¶  
END; <¶
```



5.4.2 Procedure : generate_order_summary

- Purpose: Displays total number of paid orders per customer using a cursor.

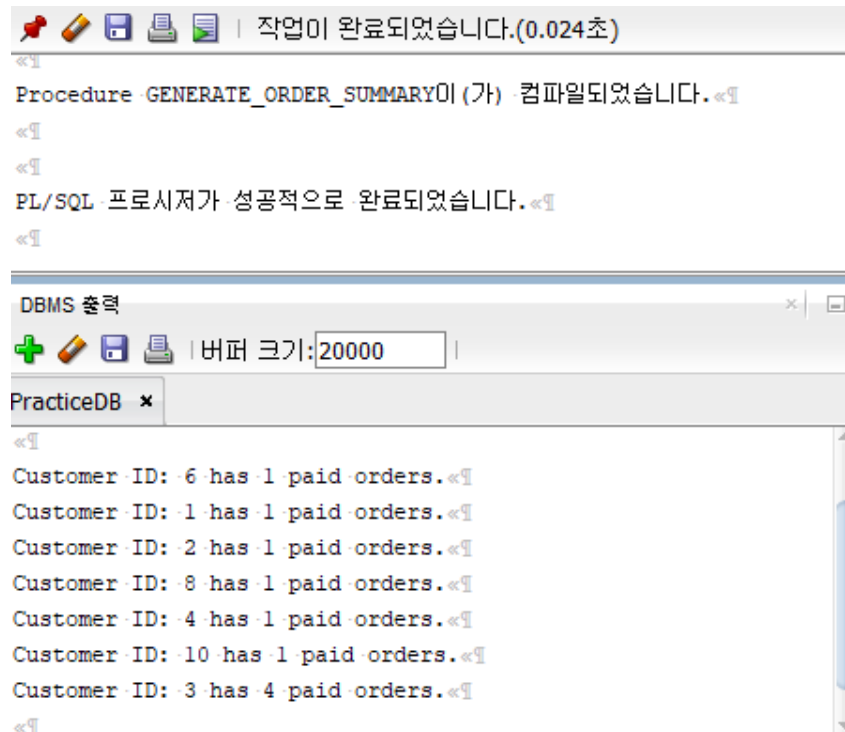
```
CREATE OR REPLACE PROCEDURE generate_order_summary IS
  CURSOR cur IS
    SELECT c.customer_id, COUNT(p.payment_id) AS total_orders
    FROM comm_customer c
    JOIN comm_shopping_cart sc ON c.customer_id = sc.customer_id
    JOIN comm_payment p ON sc.cart_id = p.cart_id
    WHERE p.payment_status = 'Paid'
    GROUP BY c.customer_id;
  v_customer_id comm_customer.customer_id%TYPE;
  v_total_orders NUMBER;
BEGIN
  OPEN cur;
  LOOP
    FETCH cur INTO v_customer_id, v_total_orders;
    EXIT WHEN cur%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('Customer-ID: ' || v_customer_id || ' has ' || v_total_orders || ' paid orders. ');
  END LOOP;
  CLOSE cur;
END;
/

```

```

BEGIN
  generate_order_summary;
END;

```



5.5. Functions

5.5.1 Function: get_customer_name

- Purpose: Returns the full name of a customer by ID.

```
CREATE OR REPLACE FUNCTION get_customer_name («  
    ... p_customer_id IN NUMBER«  
) RETURN VARCHAR2«  
IS«  
    ... v_name VARCHAR2(200);«  
BEGIN«  
    ... SELECT first_name || ' ' || last_name«  
    ... INTO v_name«  
    ... FROM comm_customer«  
    ... WHERE customer_id = p_customer_id;«  
«  
    ... RETURN v_name;«  
EXCEPTION«  
    ... WHEN NO_DATA_FOUND THEN«  
    ... RETURN 'Customer not found';«  
END;«  
/  
«  
SELECT get_customer_name(1) FROM dual;«  
«
```

	GET_CUSTOMER_NAME(1)
1	Alice Smith

5.5.2 Function: get_product_stock

- Purpose: Returns current stock quantity of a product.

```
CREATE OR REPLACE FUNCTION get_product_stock («  
    ... p_product_id IN NUMBER«  
) RETURN NUMBER«  
IS«  
    ... v_stock NUMBER;«  
BEGIN«  
    ... SELECT stock_quantity«  
    ... INTO v_stock«  
    ... FROM comm_product«  
    ... WHERE product_id = p_product_id;«  
«  
    ... RETURN v_stock;«  
EXCEPTION«  
    ... WHEN NO_DATA_FOUND THEN«  
        ... RETURN -1;«  
END;«  
/  
«  
SELECT get_product_stock(2) FROM dual;«  
~»
```

	GET_PRODUCT_STOCK(2)
1	30

5.6. Packages

- Package Name: ecommerce_utils

- Purpose: Groups related procedures and functions to support e-commerce operations like getting customer info and applying discounts.

```
CREATE OR REPLACE PACKAGE ecommerce_utils IS«¶
...FUNCTION get_customer_name(p_customer_id IN NUMBER) RETURN VARCHAR2;«¶
...FUNCTION get_product_stock(p_product_id IN NUMBER) RETURN NUMBER;«¶
«¶
...PROCEDURE apply_discount(p_product_id IN NUMBER, p_percent IN NUMBER);«¶
...PROCEDURE log_order_activity(p_cart_id IN NUMBER);«¶
END ecommerce_utils;«¶
/«¶
```

```
CREATE OR REPLACE PACKAGE BODY ecommerce_utils IS«¶
«¶
...FUNCTION get_customer_name(p_customer_id IN NUMBER) RETURN VARCHAR2 IS«¶
...v_name VARCHAR2(200);«¶
...BEGIN«¶
...SELECT first_name || ' ' || last_name INTO v_name«¶
...FROM comm_customer«¶
...WHERE customer_id = p_customer_id;«¶
...RETURN v_name;«¶
...EXCEPTION«¶
...WHEN NO_DATA_FOUND THEN RETURN 'Customer not found';«¶
...END;«¶
«¶
...FUNCTION get_product_stock(p_product_id IN NUMBER) RETURN NUMBER IS«¶
...v_stock NUMBER;«¶
...BEGIN«¶
...SELECT stock_quantity INTO v_stock«¶
...FROM comm_product«¶
...WHERE product_id = p_product_id;«¶
...RETURN v_stock;«¶
...EXCEPTION«¶
...WHEN NO_DATA_FOUND THEN RETURN -1;«¶
...END;«¶
«¶
...PROCEDURE apply_discount(p_product_id IN NUMBER, p_percent IN NUMBER) IS«¶
...BEGIN«¶
...UPDATE comm_product«¶
...SET price = price * (1 - (p_percent / 100))«¶
...WHERE product_id = p_product_id;«¶
...DBMS_OUTPUT.PUT_LINE('Discount applied.');"«¶
...END;«¶
«¶
...PROCEDURE log_order_activity(p_cart_id IN NUMBER) IS«¶
...BEGIN«¶
...DBMS_OUTPUT.PUT_LINE('Order processed for cart ID ' || p_cart_id);«¶
...END;«¶
«¶
END ecommerce_utils;«¶
/«¶
```



```
Package ECOMMERCE_UTILS이 (가) 컴파일되었습니다.<¶
```

```
<¶
```

```
<¶
```

```
Package Body ECOMMERCE_UTILS이 (가) 컴파일되었습니다.<¶
```

```
<¶
```

```
<¶  
SELECT ecommerce_utils.get_customer_name(1) FROM dual;<¶
```

```
<¶
```

```
<¶
```

스크립트 출력 x	질의 결과 x
SQL 인출된 모든 행: 1(0.017초)	
ECOMMERCE_UTILS.GET_CUSTOMER_NAME(1)	
1 Alice Smith	

```
BEGIN<¶
```

```
  ecommerce_utils.apply_discount(2, -10); ---- 10% discount<¶
```

```
END;<¶
```

```
<¶
```

DBMS 출력	x	☐
+ 버퍼 크기: 20000		
PracticeDB x		
Discount applied.<¶		
<¶		

6. Summary of PL/SQL Components

◆ Sequences

- **customer_seq**: Generates unique **customer_id** values
 - **payment_seq**: Generates unique **payment_id** values
-

◆ Indexes

- **idx_customer_email**: Speeds up customer search by email
 - **idx_product_name**: Speeds up product search by name
-

◆ Triggers

- **trg_update_product_stock**: Reduces product stock when cart is checked out
 - **trg_update_customer_stats**: Logs message when payment is marked as 'Paid'
-

◆ Procedures

- **process_complete_order**: Completes checkout and inserts payment record
 - **generate_order_summary**: Shows number of paid orders per customer
-

◆ Functions

- **get_customer_name**: Returns full name of a customer
 - **get_product_stock**: Returns stock quantity for a product
-

◆ Package: **ecommerce_utils**

- Includes 2 functions and 2 procedures
- Provides reusable operations for customer lookup, stock check, discount application, and order logging